

SQL 2

The first SQL document covered keywords, datatypes, operators and aggregate queries against a single table. This one covers the parts that involve more than one table, and the structure that makes queries fast.

A good order to learn it in:

The tables → Data constraints → JOIN queries → Sub queries
→ Indexes → Transactions

1. The tables we will use

Two tables this time, so there is something to join.

```
CREATE TABLE departments (  
  id      INT PRIMARY KEY,  
  name    VARCHAR(30) NOT NULL,  
  budget  DECIMAL(12, 2)  
);  
  
CREATE TABLE employees (  
  id          INT PRIMARY KEY,  
  name        VARCHAR(50) NOT NULL,  
  salary      DECIMAL(10, 2),  
  department_id INT,  
  manager_id  INT,  
  FOREIGN KEY (department_id) REFERENCES departments(id)  
);
```

departments :

id	name	budget
1	Engineering	500000.00
2	Sales	300000.00
3	Marketing	150000.00
4	Legal	80000.00

employees :

id	name	salary	department_id	manager_id
1	Ravi	75000.00	1	NULL
2	Amit	68000.00	1	1
3	Priya	59000.00	2	NULL
4	Neha	62000.00	2	3
5	Arjun	54000.00	3	3
6	Kiran	48000.00	NULL	1

Two gaps are there on purpose. Legal has no employees, and Kiran has no department. They show what each join type does with a row that has no match.

2. Data constraints

A constraint is a rule the database enforces on a column. The database rejects any statement that breaks one, so bad data never lands in the table.

Constraint	What it does
NOT NULL	Rejects a row where the column has no value
UNIQUE	Rejects a value already present in the column
PRIMARY KEY	Identifies each row. NOT NULL and UNIQUE together
FOREIGN KEY	Requires the value to exist in another table
CHECK	Rejects a row that fails a condition
DEFAULT	Supplies a value when the statement gives none

```
CREATE TABLE employees (
  id          INT PRIMARY KEY,
  name        VARCHAR(50) NOT NULL,
  email       VARCHAR(100) UNIQUE,
  salary      DECIMAL(10, 2) CHECK (salary > 0),
  status      VARCHAR(10) DEFAULT 'active',
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES departments(id)
);
```

Each of these statements is rejected, and the reason names the constraint:

```

INSERT INTO employees (id, name) VALUES (7, NULL);
-- name cannot be null

INSERT INTO employees (id, name) VALUES (1, 'Sara');
-- duplicate value for the primary key

INSERT INTO employees (id, name, salary) VALUES (8, 'Sara', -100);
-- check constraint on salary failed

INSERT INTO employees (id, name, department_id) VALUES (9, 'Dev', 99);
-- no department with id 99

```

PRIMARY KEY and **UNIQUE** both reject duplicates, and they differ.

	PRIMARY KEY	UNIQUE
How many per table	One	As many as needed
Accepts NULL	No	Yes
Creates an index	Yes	Yes
Meaning	This identifies the row	This value is not repeated

A foreign key also decides what happens when the row it points at is removed:

```

FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE CASCADE

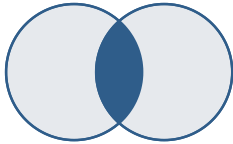
```

Option	When the parent row is deleted
RESTRICT	The delete is refused. The default in most databases
CASCADE	The child rows are deleted too
SET NULL	The child column becomes NULL

Constraint → the database refuses bad data
 Application check → only refuses the paths we remembered to check

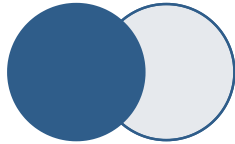
3. JOIN queries

A join reads from two tables in one query, matching rows on a column they share. **ON** says which column.



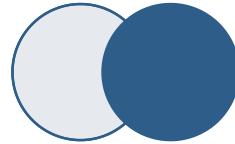
INNER JOIN

rows matching in both



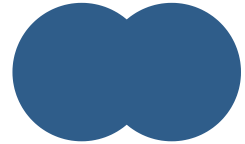
LEFT JOIN

all left, matching right



RIGHT JOIN

all right, matching left



FULL OUTER JOIN

everything from both

The left circle is the table named in FROM. The right circle is the one named in JOIN.

INNER JOIN returns only rows that match on both sides.

```
SELECT e.name, d.name AS department
FROM employees e
INNER JOIN departments d ON e.department_id = d.id
ORDER BY e.id;
```

Output:

name	department
Ravi	Engineering
Amit	Engineering
Priya	Sales
Neha	Sales
Arjun	Marketing

Kiran is missing because his `department_id` is `NULL`. Legal is missing because no employee points at it.

JOIN on its own means **INNER JOIN**.

LEFT JOIN keeps every row from the left table. Columns from the right table are `NULL` where there was no match.

```
SELECT e.name, d.name AS department
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id
ORDER BY e.id;
```

Output:

name	department
Ravi	Engineering
Amit	Engineering
Priya	Sales
Neha	Sales
Arjun	Marketing
Kiran	NULL

RIGHT JOIN keeps every row from the right table.

```
SELECT e.name, d.name AS department
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.id
ORDER BY d.id;
```

Output:

name	department
Ravi	Engineering
Amit	Engineering
Priya	Sales
Neha	Sales
Arjun	Marketing
NULL	Legal

Legal now appears with no employee. A **RIGHT JOIN** is a **LEFT JOIN** with the tables swapped, so most teams write only **LEFT JOIN**.

FULL OUTER JOIN keeps every row from both sides.

```
SELECT e.name, d.name AS department
FROM employees e
FULL OUTER JOIN departments d ON e.department_id = d.id;
```

Both gaps appear: Kiran with no department, and Legal with no employee. MySQL does not support **FULL OUTER JOIN**, so there it is written as a **LEFT JOIN** and a **RIGHT JOIN** combined with **UNION**.

A self join joins a table to itself. It is how a hierarchy inside one table is read, and it needs an alias for each side.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id
ORDER BY e.id;
```

Output:

```
employee | manager
-----+-----
Ravi     | NULL
Amit     | Ravi
Priya    | NULL
Neha     | Priya
Arjun    | Priya
Kiran    | Ravi
```

CROSS JOIN pairs every row with every row and takes no **ON**. Six employees and four departments give twenty-four rows. It is rarely what we want, and an accidental one is usually a join with the **ON** left out.

Join	Returns
INNER JOIN	Rows matching on both sides
LEFT JOIN	All left rows, plus matches from the right
RIGHT JOIN	All right rows, plus matches from the left
FULL OUTER JOIN	Everything from both sides
SELF JOIN	A table joined to itself, through two aliases
CROSS JOIN	Every combination of rows

Filter in **WHERE** → after the join, drops unmatched rows too
 Filter in **ON** → while joining, keeps the outer rows

That difference is a common mistake: putting a condition on the right table in **WHERE** turns a **LEFT JOIN** back into an **INNER JOIN**, because **NULL** fails the comparison.

4. Sub queries

A subquery is a **SELECT** inside another statement. The inner query runs first and its result feeds the outer one.

A scalar subquery returns one value, so it can sit beside a comparison operator.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

The average is 61000.00.

Output:

name	salary
Ravi	75000.00
Amit	68000.00
Neha	62000.00

A multi-row subquery returns a column of values, used with **IN**.

```
SELECT name
FROM employees
WHERE department_id IN (
    SELECT id FROM departments WHERE name IN ('Sales', 'Marketing')
);
```

Output:

name
Priya
Neha
Arjun

A correlated subquery reads a column from the outer query, so it runs once per outer row.

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(s.salary)
    FROM employees s
    WHERE s.department_id = e.department_id
);
```

Output:

name	salary
Ravi	75000.00
Neha	62000.00

Each employee is compared against the average of their own department, not the whole company. Kiran is excluded: his department is `NULL`, so the inner query matches no rows and returns `NULL`, and a comparison with `NULL` is never true.

EXISTS asks whether the subquery returns anything at all. It stops at the first row it finds.

```
SELECT d.name
FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.department_id = d.id);
```

Output:

name
Engineering
Sales
Marketing

Legal is absent, because no employee belongs to it. `SELECT 1` is the normal form here, since `EXISTS` cares only whether a row came back.

A subquery in `FROM` is a derived table. It needs a name.

```
SELECT department_id, headcount
FROM (
    SELECT department_id, COUNT(*) AS headcount
    FROM employees
    WHERE department_id IS NOT NULL
    GROUP BY department_id
) AS counts
WHERE headcount > 1
ORDER BY department_id;
```

Output:

department_id	headcount
1	2
2	2

Subquery or join? A join is usually faster, because the database can plan both tables together. A subquery is often easier to read. Modern planners rewrite many subqueries into joins on their own, so start with whichever is clearer and check the query plan if it is slow.

Subquery → one question inside another
Join → two tables read side by side

5. Indexes

An index is a separate structure that points at rows, so the database can find them without reading the whole table. It works like the index at the back of a book: we look up the term rather than turning every page.

```
CREATE INDEX idx_employees_department ON employees(department_id);
```

Without it, `WHERE department_id = 1` reads all six rows. On a table of a million rows it reads a million. With it, the database walks the index and touches only the matching rows.

`EXPLAIN` shows which one is happening:

```
EXPLAIN SELECT * FROM employees WHERE department_id = 1;
```

The word to look for is a full table scan, sometimes written `Seq Scan` or `type: ALL`, against an index scan.

Kind	What it is
Single column	One column, the common case
Composite	Several columns, in a fixed order
Unique	Also rejects duplicate values
Primary key	Made automatically for the primary key

A composite index is used left to right. This is the leftmost prefix rule, and it is asked about often.

```
CREATE INDEX idx_emp_dept_salary ON employees(department_id, salary);
```

Query	Uses the index
<code>WHERE department_id = 1</code>	Yes
<code>WHERE department_id = 1 AND salary > 60000</code>	Yes
<code>WHERE salary > 60000</code>	No, the first column is missing

An index on `(a, b)` covers queries on `a`, and on `a` with `b`. It does not cover a query on `b` alone.

Indexes have a cost. Every `INSERT`, `UPDATE` and `DELETE` has to update every index on the table, and each one takes disk space. An index that no query uses costs writes and returns nothing.

Two common reasons an index is ignored even though it exists:

```
-- Not used: the column is wrapped in a function
SELECT * FROM employees WHERE UPPER(name) = 'RAVI';

-- Not used: the pattern starts with a wildcard
SELECT * FROM employees WHERE name LIKE '%vi';

-- Used
SELECT * FROM employees WHERE name = 'Ravi';
```

Index → faster reads, slower writes

Worth indexing: columns in `WHERE`, in `JOIN ... ON`, and in `ORDER BY`. Rarely worth it: small tables, and columns with few distinct values, where the database reads the table anyway.

6. Transactions

A transaction is a group of statements the database treats as one unit. Either every statement takes effect, or none of them does.

Moving 10000 of budget from Engineering to Sales takes two statements:

```
UPDATE departments SET budget = budget - 10000 WHERE id = 1;
UPDATE departments SET budget = budget + 10000 WHERE id = 2;
```

If the database stops between them, Engineering has lost the money and Sales never received it. Neither statement is wrong on its own, and the total is now short by 10000.

ACID

ACID names the four guarantees a transaction gives.



Atomicity

Every statement takes effect, or none does



Consistency

The data always obeys its rules



Isolation

Concurrent transactions do not see unfinished work



Durability

Once committed, it survives a crash

Atomicity, consistency and durability are either kept or not. Isolation has levels, covered further down.

Atomicity. The transfer above either moves the budget or leaves both rows untouched. There is no state where one side changed and the other did not.

Consistency. Every constraint still holds when the transaction ends. With `CHECK (budget >= 0)` on the column, a transfer that would take a department below zero is refused and the whole transaction is undone.

Isolation. Two transfers running at the same time do not read each other's half-finished work. How strictly this holds is a setting.

Durability. Once the database says the transaction committed, the change is on disk. A power cut a second later does not lose it. The write-ahead log is what makes this work: the change is recorded before it is applied, so the database can finish the job after a restart.

Atomicity → all of it, or none of it

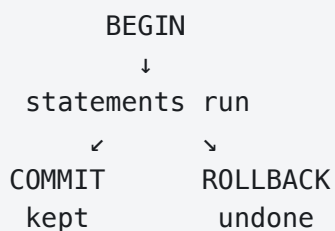
Consistency → the rules still hold afterwards

Isolation → others do not see it until it is done

Durability → once done, it stays done

BEGIN, COMMIT and ROLLBACK

Keyword	What it does
<code>BEGIN</code>	Starts a transaction. Also written <code>START TRANSACTION</code>
<code>COMMIT</code>	Keeps everything done since the transaction started
<code>ROLLBACK</code>	Undoes everything done since the transaction started



A transfer that works:

```
BEGIN;  
  
UPDATE departments SET budget = budget - 10000 WHERE id = 1;  
UPDATE departments SET budget = budget + 10000 WHERE id = 2;  
  
COMMIT;
```

Afterwards:

id	name	budget
1	Engineering	490000.00
2	Sales	310000.00

Replacing `COMMIT` with `ROLLBACK` leaves both rows exactly as they started.

Without `BEGIN`, every statement is its own transaction. Most databases run in autocommit mode, so a single `UPDATE` commits the moment it succeeds. That is why the two-statement transfer is unsafe on its own: the first `UPDATE` is already permanent before the second one runs.

Autocommit on	→ one statement, one transaction
<code>BEGIN</code>	→ many statements, one transaction

SAVEPOINT

A savepoint marks a point inside a transaction, so we can undo back to it without undoing everything.

Keyword	What it does
<code>SAVEPOINT name</code>	Marks a point to return to
<code>ROLLBACK TO SAVEPOINT name</code>	Undoes work done after the mark
<code>RELEASE SAVEPOINT name</code>	Removes the mark and keeps the work

```

BEGIN;

UPDATE departments SET budget = budget - 10000 WHERE id = 1;
UPDATE departments SET budget = budget + 10000 WHERE id = 2;

SAVEPOINT transfer_done;

UPDATE departments SET budget = budget - 500 WHERE id = 1;    -- a fee
-- the fee was wrong

ROLLBACK TO SAVEPOINT transfer_done;

COMMIT;

```

The transfer was kept and the fee was not. A plain `ROLLBACK` would have undone both.

```

ROLLBACK          → undo the whole transaction
ROLLBACK TO SAVEPOINT → undo part of it, transaction stays open

```

Isolation levels

Isolation is the only part of ACID with a setting. A stricter level prevents more problems and allows less work to run at the same time.

A dirty read is reading a change that was never committed.

Time	Session A	Session B
1	BEGIN;	
2	UPDATE employees	
	SET salary = 90000 WHERE id = 1;	
3		BEGIN;
4		SELECT salary WHERE id = 1;
		-- reads 90000
5	ROLLBACK;	
6		-- B acted on a number that
		-- never existed

A non-repeatable read is the same row giving two different answers.

Time	Session A	Session B
1		BEGIN;
2		SELECT salary WHERE id = 1; -- reads 75000
3	UPDATE employees SET salary = 80000 WHERE id = 1;	
4	COMMIT;	
5		SELECT salary WHERE id = 1; -- reads 80000, same query

A phantom read is the same query returning a different set of rows.

Time	Session A	Session B
1		BEGIN;
2		SELECT COUNT(*) FROM employees WHERE salary > 60000; -- 3 rows
3	INSERT INTO employees VALUES (7, 'Sara', 71000, 1, 1);	
4	COMMIT;	
5		SELECT COUNT(*) FROM employees WHERE salary > 60000; -- 4 rows

The four levels each prevent more of these:

Level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

The default is not the same everywhere. PostgreSQL, Oracle and SQL Server default to `READ COMMITTED`. MySQL with InnoDB defaults to `REPEATABLE READ`, and its version also prevents phantom reads, which the standard does not require.

`SERIALIZABLE` makes transactions behave as though they ran one after another. It is the safest and the slowest, worth it for money movement and rarely worth it for a reporting query.

Deadlocks

A deadlock is two transactions each holding a row the other is waiting for. Neither can move.

Time	Session A	Session B
1	BEGIN;	BEGIN;
2	UPDATE employees SET salary = 76000 WHERE id = 1;	
3		UPDATE employees SET salary = 69000 WHERE id = 2;
4	UPDATE employees SET salary = 69000 WHERE id = 2; -- waits for B	
5		UPDATE employees SET salary = 76000 WHERE id = 1; -- waits for A

The database detects this, chooses one transaction and rolls it back with a deadlock error. The other one continues.

Two things reduce them. Lock rows in the same order everywhere, so two transactions never take them in opposite order. And keep transactions short, because a transaction holds its locks until it ends.

Deadlock → both waiting, neither finishing
Fix → same lock order, shorter transactions

The pieces together

Topic	What it gives us
NOT NULL , CHECK	The database refuses bad values
PRIMARY KEY	Every row can be identified
FOREIGN KEY	A reference always points at a row that exists
INNER JOIN	Rows present in both tables
LEFT JOIN	Every row of one table, matched where possible
Self join	A hierarchy stored in one table
Scalar subquery	One value computed for the outer query
Correlated subquery	A value computed per outer row
EXISTS	Whether any matching row is there
Index	Finding rows without reading them all

Topic	What it gives us
<code>BEGIN</code> , <code>COMMIT</code>	Several statements kept as one unit
<code>ROLLBACK</code>	Undoing everything since <code>BEGIN</code>
<code>SAVEPOINT</code>	Undoing part of a transaction
Isolation level	How strictly concurrent transactions are separated

For example, imagine a payroll application. `employees.department_id` is a foreign key, so no employee can belong to a department that was deleted. `CHECK (salary > 0)` stops a negative salary. The monthly report is a `LEFT JOIN` from `departments` , so a department with nobody in it still appears with a zero. The reporting screen filters by department, so `department_id` carries an index.

The most useful thing at this point is not to memorize the join types. Take one `LEFT JOIN` from a real query and move a condition on the right table from `ON` into `WHERE` , so the unmatched rows disappear. Then open two sessions, run the dirty read timeline at `READ UNCOMMITTED` and again at `READ COMMITTED` , and see the second one refuse to show the uncommitted number.